Page 1: What is Zephyr RTOS?

Introduction, Purpose, Architecture & References

1.1 What is an RTOS?

Deterministic Response: Unlike general-purpose OSs (Windows/Linux) designed for throughput, an RTOS guarantees real-time deadlines for hardware interrupts and sensor events.

Preemptive Multitasking: Replaces wasteful CPU counting loops with threads that sleep (`k_msleep`), allowing the MCU to enter low-power idle states.

1.2 About the Zephyr Project

Open Source Governance: Hosted by the **Linux Foundation** under the permissive **Apache 2.0** license.

Industry Consortium: Governed by Microchip, Google, Intel, Nordic, NXP, ST, and Qualcomm.

Ultra-Scalable: Runs efficiently on microcontrollers with 8 KB of RAM up to multi-core MPUs.

1.3 Core Architectural Pillars

- **Devicetree (DTS):** Decouples hardware pins from C code. Switching boards requires zero code changes.
- **Kconfig System:** Granular compile-time feature selection to keep binaries lightweight.
- **Unified Driver API:** Standardized functions (`gpio_pin_set_dt`) across all microcontroller vendors.
- **Native Stacks:** Certified Bluetooth Low Energy (BLE), Wi-Fi, Ethernet, CAN-FD, USB, and TCP/IP.

1.4 Official Reference Links

Portal	Official URL
Zephyr Home Page	**https://zephyrproject.org/**
Official Documentation	**https://docs.zephyrproject.org/**
Zephyr GitHub Source	**https://github.com/zephyrproject-rtos/zephyr**

Page 2: Microchip & Zephyr Integration

How Microchip Silicon is Tied Directly into Upstream Zephyr

2.1 How Microchip Brings Zephyr to Silicon

Upstream Collaboration: Microchip Technology directly integrates and maintains official silicon support in standard Zephyr releases—no out-of-tree vendor patches required!

Seamless Upgrades: Firmware developed on Microchip Zephyr boards gains automatic access to new kernel features, security patches, and connectivity stacks.

2.2 Layered Board Porting Architecture

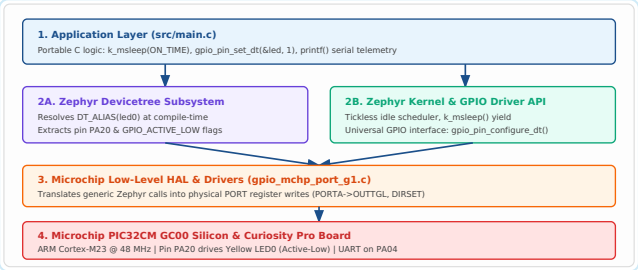


Figure 2.1: Microchip Upstream HAL & Zephyr Driver Layering.

- `modules/hal/microchip`: Vendor CMSIS headers, register definitions, and peripheral bitmasks.
- `zephyr/soc/microchip`: System clock control (48 MHz FDPLL96M), Flash wait states, and power modes.
- `zephyr/drivers`: Microchip PORT (GPIO), SERCOM (UART/I2C/SPI), and ARM SysTick timer.
- `zephyr/boards/...`: Board Devicetree definitions mapping LED0 (PA20) and SW0 (PB07).

2.3 Referral Links & Upstream Repositories

Component	Upstream Referral Link
Microchip HAL Module	**https://github.com/zephyrproject-rtos/hal_microchip**
Microchip 32-bit MCUs	**https://www.microchip.com/pic32**
Board Documentation	**https://docs.zephyrproject.org/.../pic32cm_gc00_cpro**

Page 3: Board Specifications & Peripherals

Hardware Specs & Peripheral Support for PIC32CM GC00 Curiosity Pro

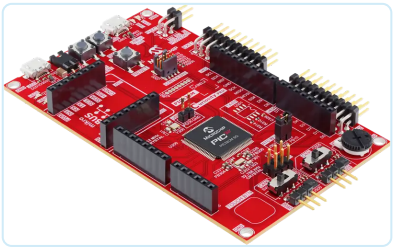


Figure 3.1: Microchip PIC32CM GC00 Curiosity Pro Evaluation Kit.

3.1 Hardware Specifications

Specification	Hardware Value	Student Takeaway
Microcontroller	PIC32CM5112GC00100	Microchip 32-bit MCU in 100-pin TQFP package
Core & Clock	ARM Cortex-M23 @ 48 MHz	Low-power ARMv8-M architecture with FDPLL96M
Flash & SRAM	512 KB Flash / 128 KB SRAM	Ample room for RTOS kernel, stacks, and user code
Onboard Debugger	Microchip nEDBG	CMSIS-DAP programmer & CDC Virtual COM Port

3.2 Available Peripheral Support in Zephyr

Peripheral	Zephyr Driver	Support Status	Hardware Mapping
GPIO	`gpio_mchp_port_g1`	**Supported**	PA20 (LED0), PA18 (LED1), PB07 (SW0)
UART Console	`uart_mchp_sercom`	**Supported**	PA04 / PA05 (115200 8N1 via USB COM)
Pin Multiplexing	`pinctrl-pic32c`	**Supported**	Devicetree pin routing controller
System Timer	ARM Cortex-M SysTick	**Supported**	1000 Hz kernel tick for `k_msleep()`
Flash NVM	`nvmctrl`	**Supported**	Internal code partition (496KB)

Page 4: Development Platform

Hardware and Software Architecture Overview

01 Hardware

- **Board:** PIC32CM GC00 Curiosity Pro (EV33A17A)
- **MCU:** PIC32CM5112GC00100 (32-bit ARM Cortex-M23 @ 48 MHz)
- **Onboard LED:** Yellow LED0 (Pin PA20, Active-Low) for application demonstration
- **USB Connection:** Micro-USB (J100) for programming and interactive debugging

02 Software

- **MPLAB VS Code:** Modern IDE workspace with Zephyr and C/C++ extensions
- **Zephyr RTOS Workspace:** Portable real-time operating system kernel
- **West Build Tool:** Automated CMake, Ninja, and runner meta-tool
- **Compatible Zephyr Toolchain:** ARM Embedded SDK (`arm-zephyr-eabi`)

Outcome: A working environment for building and running Zephyr applications on Microchip hardware.

Page 5: Cloning & Local Setup

How to Clone the Zephyr Project on Your Local Computer

Step 1: Create Workspace & Virtual Environment

```
mkdir -p ~/zephyrproject && cd ~/zephyrproject
python3 -m venv .venv
source .venv/bin/activate
```

Step 2: Install West (Zephyr's Meta-Tool)

```
pip install --upgrade pip
pip install west
```

Step 3: Initialize Zephyr & Download Modules

```
# Initialize west manifest in local directory
west init -l zephyr

# Pull Microchip HAL, CMSIS, and module dependencies
west update
```

Step 4: Export CMake & Install Python Requirements

```
west zephyr-export
pip install -r zephyr/scripts/requirements.txt
```

Remember: Always run `source ~/zephyrproject/.venv/bin/activate` when opening a new terminal!

Page 6: Environment & Toolchain Setup

Installing System Packages, ARM SDK & USB Permissions

Step 1: Install Host Operating System Packages

```
sudo apt update && sudo apt install -y git cmake ninja-build gperf ccache dfu-util
device-tree-compiler wget python3-dev python3-pip xz-utils make gcc
```

Step 2: Install the Zephyr ARM SDK Cross-Compiler

```
cd ~
wget https://github.com/zephyrproject-rtos/sdk-ng/releases/download/v0.16.8/zephyr-
sdk-0.16.8_linux-x86_64.tar.xz
tar xvf zephyr-sdk-0.16.8_linux-x86_64.tar.xz
cd zephyr-sdk-0.16.8 && ./setup.sh -t arm-zephyr-eabi -h -c
```

Step 3: Setup USB udev Rules for Microchip Debugger

```
sudo cp ~/zephyrproject/zephyr/scripts/udev/rules.d/* /etc/udev/rules.d/
sudo udevadm control --reload-rules && sudo udevadm trigger
```

Quick Verification Checklist

Command	Expected Output
`west --version`	West version 1.2.0 or newer
`arm-zephyr-eabi-gcc --version`	GCC 12.2.0 (Zephyr SDK)

Page 7: Application Configuration Files

Understanding CMakeLists.txt, prj.conf, and app.overlay

7.1 CMakeLists.txt — Project Build Script

```
cmake_minimum_required(VERSION 3.20.0)
find_package(Zephyr REQUIRED HINTS $ENV{ZEPHYR_BASE})
project(pic32cm_gc00_blinky)
target_sources(app PRIVATE src/main.c)
```

What it does: Finds Zephyr packages, loads the board Devicetree, links toolchains, and registers `src/main.c` for compilation.

7.2 prj.conf — Kconfig Feature Flags

```
CONFIG_GPIO=y                # Enables Microchip PORT GPIO driver
CONFIG_PRINTK=y              # Enables kernel printk() logging
CONFIG_STDOUT_CONSOLE=y      # Directs C printf() to the serial port
```

What it does: Zephyr disables drivers by default. `prj.conf` turns on only the drivers your application needs, saving memory.

7.3 app.overlay — Custom Devicetree Overlays

```
/ {
    aliases {
        my-led = &custom_led;
    };
    leds {
        compatible = "gpio-leds";
        custom_led: led_custom {
            gpios = <&porta 18 GPIO_ACTIVE_LOW>;
            label = "Secondary LED1";
        };
    };
};
```

What it does: Allows students to remap pins or add custom peripherals without modifying the base board files.

Page 8: Building & Flashing Firmware

Step-by-Step West Compilation and USB Flashing Guide

8.1 The Build Pipeline

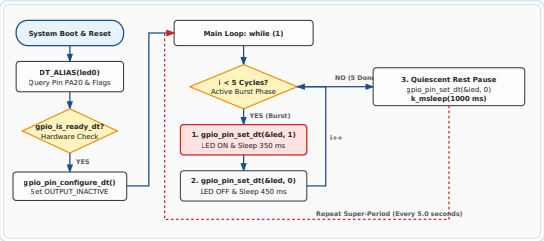


Figure 8.1: Zephyr Compilation & Execution Flow.

8.2 Build Commands

Clean Pristine Build:

```
west build -b pic32cm_gc00_cpro pic32cm_gc00_blinky -p always
```

Incremental Fast Build:

```
west build -p auto
```

8.3 Flashing to Hardware

Connect micro-USB to the **DEBUG USB** port (J100) and run:

```
west flash
```

Programs internal Flash over USB via the onboard nEDBG debugger.

8.4 Generated Output Binaries

File	Location	Description
zephyr.elf	build/zephyr/	Full binary with symbols for debugging
zephyr.bin	build/zephyr/	Flat machine code burned to MCU Flash

Page 9: Serial Telemetry & Timing

UART Output Logs and Deterministic Waveform Analysis

9.1 Serial Console Setup (115200 8N1)

```
# Connect via picocom on Linux (or PuTTY on Windows COMx)
picocom -b 115200 /dev/ttyACM0
```

9.2 Expected Serial Output Log

```
*** Booting Zephyr OS build v3.7.0 ***
Iteration 1/5 - LED state: ON (350 ms ON)
Iteration 1/5 - LED state: OFF (450 ms OFF)
Iteration 2/5 - LED state: ON (350 ms ON)
Iteration 2/5 - LED state: OFF (450 ms OFF)
Iteration 3/5 - LED state: ON (350 ms ON)
Iteration 3/5 - LED state: OFF (450 ms OFF)
Iteration 4/5 - LED state: ON (350 ms ON)
Iteration 4/5 - LED state: OFF (450 ms OFF)
Iteration 5/5 - LED state: ON (350 ms ON)
Iteration 5/5 - LED state: OFF (450 ms OFF)
Pause - LED state: OFF (1000 ms Pause)
```

9.3 Timing Waveform Diagram

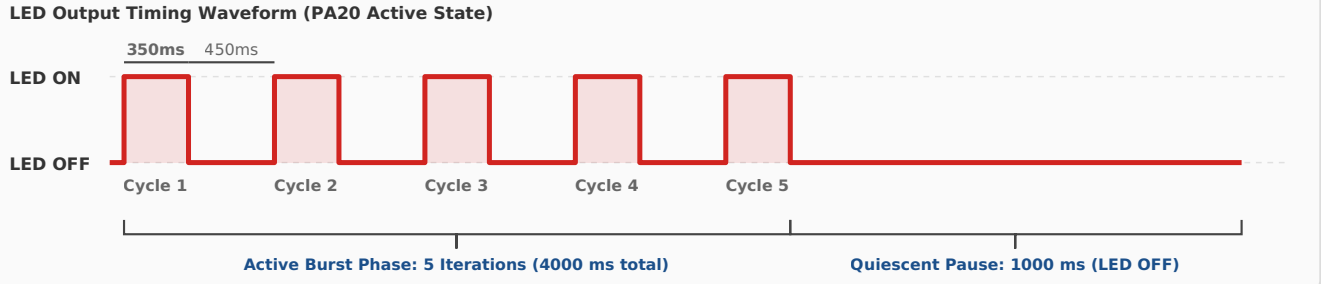


Figure 9.1: Deterministic 5-Burst Active Pulse & 1000 ms Quiescent Rest Waveform.

Page 10: Conclusion & Lab Exercises

Summary, Student Challenges & References

10.1 Key Learning Summary

- **Portability:** Standardized APIs (`gpio_pin_set_dt`, `k_msleep`) run unchanged on any Zephyr MCU.
- **Devicetree Abstraction:** Pin numbers and polarity flags (`GPIO_ACTIVE_LOW`) are decoupled from C code.
- **Tickless Power Savings:** `k_msleep()` allows the ARM Cortex-M23 core to sleep between blinks.

10.2 Hands-On Student Lab Challenges

Challenge 1 (Morse Code SOS): Modify delay macros to pulse international SOS: 3 short (150ms), 3 long (450ms), 3 short (150ms), followed by a 2000ms rest pause.

Challenge 2 (Dual Alternating Beacon): Add secondary LED1 (PA18) via `DT_ALIAS(led1)` and alternate like an emergency responder light bar.

Challenge 3 (Pushbutton Gate): Read switch SW0 (PB07) via `DT_ALIAS(sw0)`. Only run the 5-burst blinking pattern when SW0 is held down.

10.3 Quick Resource Links

Resource	URL
Microchip Official	**https://www.microchip.com**
Zephyr Project	**https://zephyrproject.org**
Project GitHub	**https://github.com/sujandurai/pic32cm_gc00_blinky**

Congratulations on mastering Zephyr RTOS fundamentals on Microchip PIC32CM hardware!